

Guia de Implementacao

Guardrails de Custo de IA em Dev para Azure AI Foundry

Como dar a um desenvolvedor um Resource Group com teto de US\$ 500 para testar IA sem estourar o orcamento

Repositorio: [azure-foundry-cost-guardrails](#)

Versao 1.0 | Generico e agnostico de ambiente | PowerShell 7 + Azure CLI

Conteudo

1. O problema	3
2. A arquitetura: defesa por tempo de reacao.....	3
3. O que queima os US\$ 500	4
4. IF: qual Foundry voce tem?	4
5. A matematica do US\$ 500.....	5
6. Estrutura do repositorio	5
7. Pre-requisitos	5
8. O que voce precisa editar.....	6
9. Passo a passo de implementacao.....	6
10. Os scripts (agnosticos).....	9
scripts/apply-guardrails.ps1	9
scripts/verify-guardrails.ps1	13
scripts/teardown-guardrails.ps1	13
11. As policies e o custom role.....	15
policies/definitions/deny-deployment-sku.json (a trava central).....	15
rbac/role-ai-sandbox-safe-dev.json	16
policies/params/allowed-resource-types.accountbased.json	17
12. Apendice: policies built-in usadas.....	18
Apendice B: custom de hub-based (deny GPU compute)	18
Aviso	19

1. O problema

A premissa é simples: você dá a um desenvolvedor um Resource Group e US\$ 500 para ele testar IA no Azure AI Foundry. Duas regras inegociáveis. Primeira: ele não pode estourar o orçamento. Segunda: se estourar, você precisa estancar o mais rápido possível.

O furo clássico não é consumo de token. É provisionamento por relógio. Um modelo provisionado (PTU), um modelo Fireworks/BYOM ou um compute GPU cobram por hora reservada, mesmo que ninguém gere um único token. Foi exatamente esse o incidente: um dev subiu capacidade provisionada e o custo correu sozinho, sem consumo.

A verdade incomoda: o Azure não tem um botão nativo de "pare em US\$ 500" para um Resource Group. O Budget é alerta, não freio. O hard-stop por gasto só existe em ofertas Dev/Test/MSDN e no nível de subscription. Então a estratégia não é confiar num freio mágico: é limitar o quanto se pode gastar por unidade de tempo na origem (Camada 1) e ter um teto duro de token no gateway (Camada 3).

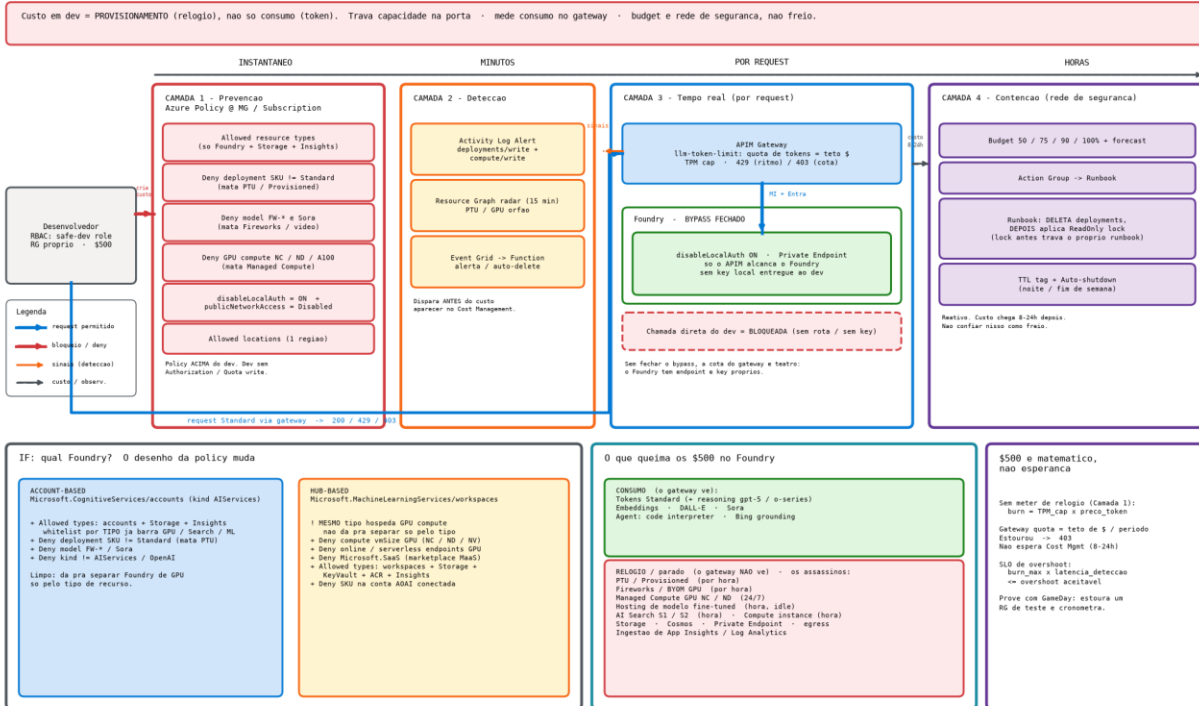
2. A arquitetura: defesa por tempo de reação

O desenho tem quatro camadas, organizadas pela velocidade com que cada uma reage. Quanto mais à esquerda, mais cedo a defesa atua. A ideia é nunca depender de uma camada sozinha.

Camada	Quando age	Mecanismo	O que faz
1. Prevenção	Instantâneo	Azure Policy + RBAC (custom role)	Barra na porta o que cobra por relógio: PTU/provisionado, GPU compute, serverless/marketplace. O dev nem cria.
2. Detecção	Minutos	Activity Log Alert + Resource Graph + Event Grid	Avisa (ou deleta) antes do custo aparecer no Cost Management.
3. Tempo real	Por request	APIM + policy llm-token-limit	Quota de token por time = teto em US\$. Retorna 403 ao estourar. O TPM (429) limita o ritmo.
4. Contenção	Horas	Budget + Action Group + Runbook	Rede de segurança reativa: deleta deployments e aplica lock.

Topologia completa (a caixa de allow-list de modelos aparece como opcional):

Governança de Custo de IA em Dev · RG do desenvolvedor com teto de \$500



3. O que queima os US\$ 500

Metade do gasto o gateway ve (consumo por token). A outra metade ele nao ve, porque e cobranca por relógio. Por isso a Camada 1 (Policy) e o que de fato salva o orcamento: ela mata na origem o que o gateway jamais enxerga.

Consumo (o gateway VE)	Relógio / parado (o gateway NAO ve)
Tokens de prompt e completion	PTU / Provisioned (cobra por hora)
Tokens de reasoning (gpt-5 / o-series), invisíveis no texto	Fireworks / BYOM GPU (por hora)
Embeddings	Managed Compute GPU NC / ND rodando 24/7
DALL-E, Sora (video)	Hosting de modelo fine-tuned (parado)
Agent: code interpreter e Bing grounding	AI Search S1 / S2, compute instance
	Storage, Cosmos, Private Endpoint, egress, ingestao de observabilidade

4. IF: qual Foundry voce tem?

O desenho da policy muda conforme o tipo do Foundry. Descubra o seu antes de aplicar:

```
az resource list --resource-type Microsoft.CognitiveServices/accounts --query "[].{nome:name, kind:kind}" -o table
az resource list --resource-type Microsoft.MachineLearningServices/workspaces -o table
```

Cenario	Tipo de recurso	Implicacao
account-based	Microsoft.CognitiveServices/accounts (kind AIServices)	Separacao limpa por tipo. A allow-list de tipos ja barra GPU/ML. Use -Scenario accountbased.
hub-based	Microsoft.MachineLearningServices/workspaces	O MESMO tipo hospeda GPU compute. Precisa dos denies extras de vmSize GPU e serverless/marketplace. Use -Scenario hubbased.

5. A matematica do US\$ 500

Como nao existe stop-at-500 nativo, o teto vira gestao de risco. A logica:

- Sem meter de relógio (a Camada 1 mata PTU/GPU), o burn vira funcao de token: burn por minuto = $TPM_cap \times preco_por_token$. Voce controla o pior caso escolhendo o TPM.
- Teto duro de gasto = quota de token do gateway (Camada 3). Ao estourar, 403. Isso independe do lag do Cost Management (8 a 24h).
- Overshoot maximo = $burn_max \times latencia_de_deteccao$. Esse e o numero honesto para levar ao cliente.
- Prove num GameDay: estoure de proposito um RG de teste e cronometre quanto tempo leva para estancar.

6. Estrutura do repositorio

```

azure-foundry-cost-guardrails/
  README.md
  LICENSE
  config/
    example.parameters.json
  docs/
    architecture.md
    builtins-reference.md
    topology.excalidraw
    topology.png
  policies/
    definitions/
      deny-deployment-sku.json           (CUSTOM: mata PTU/provisionado)
      deny-gpu-mlcompute.json           (hub-based)
      deny-serverless-marketplace.json  (hub-based)
    params/
      allowed-resource-types.accountbased.json
      allowed-resource-types.hubbased.json
      allowed-locations.json
      approved-models.json              (opcional)
  rbac/
    role-ai-sandbox-safe-dev.json
  scripts/
    apply-guardrails.ps1
    verify-guardrails.ps1
    teardown-guardrails.ps1

```

7. Pre-requisitos

- Azure CLI instalado e logado (az login) no tenant do cliente.
- PowerShell 7 ou superior.

- Permissao no escopo da subscription: Owner OU (Resource Policy Contributor + User Access Administrator). Necessario para criar policy definitions, assignments e o custom role.
- Definir o cenario (account-based ou hub-based), o nome do RG sandbox e a regioao permitida.

8. O que voce precisa editar

Nada de subscription, tenant ou nome de recurso fica escrito no codigo. Tudo entra por parametro ou pelo arquivo de configuracao. O unico lugar com placeholder e o config, que voce copia e preenche:

```
{
  "subscriptionId": "<SUA_SUBSCRIPTION_ID>",
  "resourceGroup": "rg-dev-sandbox-01",
  "location": "eastus",
  "scenario": "accountbased",
  "effect": "Audit",
  "allowedSkus": ["Standard", "GlobalStandard", "DataZoneStandard"],
  "enableApprovedModels": false,
  "allowedPublishers": ["OpenAI", "Microsoft"]
}
```

Parametro / arquivo	O que e	Exemplo
-SubscriptionId	Subscription alvo (vem no comando, nao no codigo)	00000000-0000-0000-0000-000000000000
-ResourceGroup	RG sandbox do desenvolvedor	rg-dev-sandbox-01
-Location	Regiao permitida	eastus
-Scenario	account-based ou hub-based	accountbased
-Effect	Audit primeiro, depois Deny	Audit
policies/params/allowed-resource-types.*.json	Tipos de recurso que o dev pode criar	edite a lista
-AllowedSkus	SKUs de consumo por token permitidos	Standard, GlobalStandard, DataZoneStandard
-EnableApprovedModels	(Opcional) liga a allow-list de modelos por publisher	desligado por default

Sobre a allow-list de modelos (Fireworks/Sora): ela vem DESLIGADA por default. Sem ela, o dev pode implantar modelos de qualquer publisher, mas a trava de SKU/PTU continua barrando a forma cobrada por hora. Para voltar a bloquear publishers de terceiro, basta a flag - EnableApprovedModels e ajustar -AllowedPublishers.

9. Passo a passo de implementacao

Passo 1. Login e contexto

```
az login
az account set --subscription <SUA_SUBSCRIPTION_ID>
```

Passo 2. Aplicar em modo Audit (nao bloqueia nada)

Comece sempre em Audit. Nada e bloqueado; o Azure so registra o que SERIA negado. O - AlertEmail ja liga a Camada 2 (deteccao). Use -DryRun para so ver os comandos.

```
./scripts/apply-guardrails.ps1 -SubscriptionId <SUB_ID> -ResourceGroup rg-dev-sandbox-01 -Location eastus -
Scenario accountbased -Effect Audit -AlertEmail voce@empresa.com
```

Passo 3. Revisar a compliance

Olhe o resultado no portal (Policy > Compliance) ou rode o verify. A compliance leva alguns minutos para popular.

```
./scripts/verify-guardrails.ps1 -SubscriptionId <SUB_ID> -ResourceGroup rg-dev-sandbox-01
```

Passo 4. Virar para Deny

Confortavel com o que viu, rode de novo trocando para Deny. Agora as travas passam a bloquear de fato.

```
./scripts/apply-guardrails.ps1 -SubscriptionId <SUB_ID> -ResourceGroup rg-dev-sandbox-01 -Location eastus -
Scenario accountbased -Effect Deny -AlertEmail voce@empresa.com
```

Passo 5. Camada 2: confirmar os alertas de deteccao

Com -AlertEmail, o apply criou um Action Group e um Activity Log Alert no RG. Qualquer novo deployment de IA dispara um email em minutos, antes do custo aparecer no Cost Management. Confirme que subiu:

```
az monitor activity-log alert list --resource-group rg-dev-sandbox-01 -o table
```

Para a varredura sob demanda (a parte Resource Graph do desenho), liste todo recurso de IA no RG. Requer a extensao: az extension add -n resource-graph.

```
az graph query -q "resources | where resourceGroup =~ 'rg-dev-sandbox-01' | where type startswith
'microsoft.cognitiveservices' or type startswith 'microsoft.machinelearningservices' | project name, type,
location" -o table
```

Passo 6. Custom role: criar, atribuir e tirar o Owner do dev

Esta peca define QUEM o dev e. A ideia: ele tem SO esta role no RG, e nada mais. Ela deixa criar e gerenciar deployments Standard e ler telemetria, mas nao mexer em RBAC, quota, billing, nem pegar a chave da conta.

6a. Criar a role. O apply ja faz isso (injetando o AssignableScopes a partir da sua subscription). Para criar na mao, edite o placeholder <SUA_SUBSCRIPTION_ID> no JSON e rode:

```
az role definition create --role-definition rbac/role-ai-sandbox-safe-dev.json
# conferir que subiu:
az role definition list --name "AI Sandbox Safe Dev" -o table
```

6b. Atribuir a role ao dev, no escopo do RG (nada acima disso):

```
az role assignment create --assignee <UPN_OU_OBJECTID_DO_DEV> --role "AI Sandbox Safe Dev" --scope
/subscriptions/<SUB_ID>/resourceGroups/rg-dev-sandbox-01
```

6c. Tirar Owner/Contributor do dev. CRITICO: NotActions nao e um deny global, so subtrai o que ESTA role concede. Se o dev ainda for Owner ou Contributor (direto ou herdado da subscription), as travas da role nao valem nada. Veja a heranca e remova o que houver:

```
# ver tudo que o dev tem, inclusive herdado de cima:
az role assignment list --assignee <UPN_OU_OBJECTID_DO_DEV> --scope
/subscriptions/<SUB_ID>/resourceGroups/rg-dev-sandbox-01 --include-inherited -o table
# remover uma atribuicao ampla que exista no RG (exemplo):
az role assignment delete --assignee <UPN_OU_OBJECTID_DO_DEV> --role Contributor --scope
/subscriptions/<SUB_ID>/resourceGroups/rg-dev-sandbox-01
```

Se aparecer Owner/Contributor HERDADO da subscription ou do management group, esta role no RG nao segura, porque heranca de cima sempre vence. Nesse caso, coloque o sandbox numa subscription separada ou use Deny Assignment (Blueprints / managed).

Passo 7. Zerar a quota de provisionado e de GPU (defesa em profundidade)

A policy bloqueia a forma cobrada por hora; zerar a quota e a segunda barreira, garante que nao existe capacidade reservavel mesmo se algo escapar. Faca por regioao.

7a. Veja a quota de provisionado e os limites de IA na regioao (procure linhas OpenAI / ProvisionedManaged; em subscription nova o PTU costuma vir 0):

```
az cognitiveservices usage list --location <REGIAO> -o table
```

7b. Cenário hub-based: veja a vCPU das famílias GPU (NC / ND / NV):

```
az vm list-usage --location <REGIAO> --query "[?contains(localName,'NC') || contains(localName,'ND') || contains(localName,'NV')].{familia:localName, limite:limit, uso:currentValue}" -o table
```

7c. Para reduzir: no Azure AI Foundry, Management center > Quota (ou Azure portal > a conta de IA > Quotas), baixe o provisionado para 0 nas regiões que o dev não usa. Aumento e self-service; redução pode exigir ticket de suporte.

Passo 8. Fechar o bypass do Foundry

Se o dev tiver a chave e o endpoint público, ele fura o gateway e a quota de token vira teatro. disableLocalAuth e publicNetworkAccess são propriedades da conta; use az resource update (não e --api-properties):

```
$fid =
"/subscriptions/<SUB_ID>/resourceGroups/<RG_DO_FOUNDRY>/providers/Microsoft.CognitiveServices/accounts/<FOUNDRY>"
az resource update --ids $fid --set properties.disableLocalAuth=true
az resource update --ids $fid --set properties.publicNetworkAccess=Disabled
# depois: crie um Private Endpoint e acesse via Entra ID, sem chave
```

Passo 9. Budget + Action Group + Runbook (Camada 4)

Crie um Budget no RG com alertas em 50/75/90/100% e forecast, apontando para um Action Group que dispara um Runbook. Ordem crítica no runbook: primeiro DELETA os deployments, DEPOIS aplica o ReadOnly lock. Se inverter, o lock trava o próprio runbook.

Passo 10. (Opcional) Gateway APIM para o teto de token em US\$

Para o teto duro por time, coloque um APIM na frente do Foundry com a policy llm-token-limit (quota de token = 403, TPM = 429) e llm-emit-token-metric para o chargeback por time. E a Camada 3 completa.

Passo 11. Verificar com prova viva

Rode o verify apontando para o Foundry: ele tenta criar um deployment PTU e espera ser bloqueado.

```
./scripts/verify-guardrails.ps1 -SubscriptionId <SUB_ID> -ResourceGroup rg-dev-sandbox-01 -FoundryAccount <FOUNDRY>
```

Rollback

Para remover tudo que foi aplicado:

```
./scripts/teardown-guardrails.ps1 -SubscriptionId <SUB_ID> -ResourceGroup rg-dev-sandbox-01 -Scenario accountbased
```


10. Os scripts (agnosticos)

scripts/apply-guardrails.ps1

```
#Requires -Version 7.0
<#
    apply-guardrails.ps1
    Aplica as travas de governanca de custo de IA num Resource Group de desenvolvedor.

    Pre-requisitos:
    - Azure CLI logado: az login
    - Permissao no escopo: Owner OU (Resource Policy Contributor + User Access Administrator)
    - PowerShell 7+

    IMPORTANTE: nada e fixo no ambiente. Subscription, RG, regio e cenario vem por parametro.

    Exemplos:
    # 1) SEMPRE comece em Audit (observa, nao bloqueia nada). -AlertEmail liga a Camada 2 (deteccao).
    ./apply-guardrails.ps1 -SubscriptionId <SUB_ID> -ResourceGroup rg-dev-sandbox-01 -Location eastus -
    Scenario accountbased -Effect Audit -AlertEmail voce@empresa.com

    # 2) Revisada a compliance no portal, vire para Deny
    ./apply-guardrails.ps1 -SubscriptionId <SUB_ID> -ResourceGroup rg-dev-sandbox-01 -Location eastus -
    Scenario accountbased -Effect Deny

    # 3) (Opcional) reativar a allow-list de modelos por publisher (bloqueia Fireworks e terceiros)
    ./apply-guardrails.ps1 -SubscriptionId <SUB_ID> -ResourceGroup rg-dev-sandbox-01 -Location eastus -Effect
    Deny -EnableApprovedModels
#>
[CmdletBinding()]
param(
    [Parameter(Mandatory)] [string] $SubscriptionId,
    [Parameter(Mandatory)] [string] $ResourceGroup,
    [Parameter(Mandatory)] [string] $Location,
    [ValidateSet('accountbased','hubbased')] [string] $Scenario = 'accountbased',
    [ValidateSet('Audit','Deny','Disabled')] [string] $Effect = 'Audit',
    [string[]] $AllowedLocations = @(),
    [string[]] $AllowedSkus = @('Standard','GlobalStandard','DataZoneStandard'),
    [string[]] $AllowedPublishers = @('OpenAI','Microsoft'),
    [string] $AlertEmail = '',
    [switch] $EnableApprovedModels,
    [switch] $DryRun
)

$errorActionPreference = 'Stop'
$here = Split-Path -Parent $MyInvocation.MyCommand.Path
$repo = Split-Path -Parent $here
$subScope = "/subscriptions/$SubscriptionId"
$rgScope = "$subScope/resourceGroups/$ResourceGroup"
if (-not $AllowedLocations -or $AllowedLocations.Count -eq 0) { $AllowedLocations = @($Location) }

# IDs de policy BUILT-IN do Azure (globais, iguais em qualquer tenant)
$BUILTIN = @(
    AllowedTypes = '/providers/Microsoft.Authorization/policyDefinitions/a08ec900-254a-4555-9bf5-e42af04b5c5c'
    AllowedLocations = '/providers/Microsoft.Authorization/policyDefinitions/e56962a6-4747-49cd-b67b-bf8b01975c4c'
    DisableLocalAuth = '/providers/Microsoft.Authorization/policyDefinitions/71ef260a-8f18-47b7-abcb-62d0673d94dc'
    RestrictNetwork = '/providers/Microsoft.Authorization/policyDefinitions/037eea7a-bd0a-46c5-9a66-03aea78705d3'
    ApprovedModels = '/providers/Microsoft.Authorization/policyDefinitions/aafe3651-cb78-4f68-9f81-e7e41509110f'
)
```

```

ManagedIdentity = '/providers/Microsoft.Authorization/policyDefinitions/fe3fd216-4f83-4fc1-8984-2bbec80a3418'
}

function Invoke-Az([string[]] $CliArgs) {
    if ($DryRun) { Write-Host "DRYRUN> az $($CliArgs -join ' ')" -ForegroundColor DarkYellow; return }
    $out = az @CliArgs 2>&1
    if ($LASTEXITCODE -ne 0) { throw "Falha no az: $($CliArgs -join ' ') :: $out" }
    return $out
}

function Assign([string] $Name, [string] $PolicyId, $ParamsObj) {
    $cli = @('policy','assignment','create','--name',$Name,'--scope',$rgScope,'--policy',$PolicyId)
    $tmp = $null
    if ($ParamsObj) {
        $tmp = [System.IO.Path]::GetTempFileName()
        ($ParamsObj | ConvertTo-Json -Depth 8) | Set-Content -Path $tmp -Encoding utf8
        $cli += @('--params', $tmp)
    }
    try { Invoke-Az $cli | Out-Null; Write-Host "  assign: $Name" -ForegroundColor DarkGray }
    finally { if ($tmp) { Remove-Item $tmp -ErrorAction SilentlyContinue } }
}

# Preflight
$ctx = az account show -o json 2>$null | ConvertFrom-Json
if (-not $ctx) { throw "Rode 'az login' antes de executar." }
Invoke-Az @('account','set','--subscription',$SubscriptionId) | Out-Null
Write-Host "Cenario=$Scenario Effect=$Effect RG=$ResourceGroup Regiao=$Location" -ForegroundColor Cyan

# 1) Definicoes CUSTOM (so o que NAO tem built-in) -- criadas no nivel da subscription.
# IMPORTANTE: o az espera --rules = APENAS o policyRule e --params = APENAS as parameter
# definitions. Os arquivos do repo sao definicoes COMPLETAS (para importar no portal tambem),
# entao extraimos cada parte e gravamos em arquivos temporarios.
function DefineCustom([string] $Name, [string] $RelFile) {
    if ($DryRun) { Write-Host "DRYRUN> definir policy $Name a partir de $RelFile" -ForegroundColor DarkYellow;
return }
    $def = Get-Content (Join-Path $repo $RelFile) -Raw | ConvertFrom-Json
    $p = $def.properties
    $mode = if ($p.mode) { $p.mode } else { 'All' }
    $ruleTmp = [System.IO.Path]::GetTempFileName()
    $paramTmp = [System.IO.Path]::GetTempFileName()
    ($p.policyRule | ConvertTo-Json -Depth 30) | Set-Content -Path $ruleTmp -Encoding utf8
    ($p.parameters | ConvertTo-Json -Depth 30) | Set-Content -Path $paramTmp -Encoding utf8
    try {
        Invoke-Az @('policy','definition','create','--name',$Name,'--mode',$mode,
            '--display-name',$p.displayName,'--description',$p.description,
            '--subscription',$SubscriptionId,'--rules',$ruleTmp,'--params',$paramTmp) | Out-Null
        Write-Host "  define: $Name" -ForegroundColor DarkGray
    } finally { Remove-Item $ruleTmp,$paramTmp -ErrorAction SilentlyContinue }
}

Write-Host "=== Definicoes custom" -ForegroundColor Cyan
DefineCustom 'deny-foundry-sku-ptu' 'policies/definitions/deny-deployment-sku.json'
if ($Scenario -eq 'hubbased') {
    DefineCustom 'deny-ml-gpu-compute' 'policies/definitions/deny-gpu-mlcompute.json'
    DefineCustom 'deny-ml-serverless-mktplace' 'policies/definitions/deny-serverless-marketplace.json'
}

# 2) Allowed resource types + locations (built-ins Deny-only). Em RG novo/vazio, seguro aplicar ja.
Write-Host "=== Allowed types + locations (Deny-only)" -ForegroundColor Cyan
$typesFile = Join-Path $repo "policies/params/allowed-resource-types.$Scenario.json"
Invoke-Az @('policy','assignment','create','--name','allowed-types','--scope',$rgScope,
    '--policy',$BUILTIN.AllowedTypes,'--params',$typesFile) | Out-Null
Assign 'allowed-locations' $BUILTIN.AllowedLocations @{ listOfAllowedLocations = @{ value = $AllowedLocations
} }

```

```

# 3) Built-ins do AI Services (suportam Audit/Deny pelo parametro effect)
Write-Host "==> Built-ins AI Services (effect=$Effect)" -ForegroundColor Cyan
Assign 'deny-localauth' $BUILTIN.DisableLocalAuth @{ effect = @{ value = $Effect } }
Assign 'restrict-network' $BUILTIN.RestrictNetwork @{ effect = @{ value = $Effect } }
Assign 'require-mi' $BUILTIN.ManagedIdentity @{ effect = @{ value = $Effect } }

# approved-models e OPCIONAL (off por default). Quando ligado, bloqueia Fireworks e publishers de terceiro.
if ($EnableApprovedModels) {
    Assign 'approved-models' $BUILTIN.ApprovedModels @{
        effect = @{ value = $Effect }
        allowedPublishers = @{ value = $AllowedPublishers }
        allowedAssetIds = @{ value = @() }
    }
} else {
    Write-Host " approved-models DESLIGADO (terceiros liberados; a trava de SKU/PTU ainda barra a forma por hora)" -ForegroundColor Yellow
}

# 4) Custom: trava de custo central (SKU/PTU) + customs de hub-based
Write-Host "==> Assignments custom (effect=$Effect)" -ForegroundColor Cyan
function DefId([string] $n) { "$subScope/providers/Microsoft.Authorization/policyDefinitions/$n" }
Assign 'deny-sku-ptu' (DefId 'deny-foundry-sku-ptu') @{ effect = @{ value = $Effect }; allowedSkus = @{ value = $AllowedSkus } }
if ($Scenario -eq 'hubbased') {
    Assign 'deny-gpu-compute' (DefId 'deny-ml-gpu-compute') @{ effect = @{ value = $Effect } }
    Assign 'deny-serverless' (DefId 'deny-ml-serverless-mktplace') @{ effect = @{ value = $Effect } }
}

# 5) Custom role safe-dev -- injeta o AssignableScope a partir da subscription (nada fixo no JSON)
Write-Host "==> Custom role 'AI Sandbox Safe Dev'" -ForegroundColor Cyan
if (-not $DryRun) {
    $role = Get-Content (Join-Path $repo 'rbac/role-ai-sandbox-safe-dev.json') -Raw | ConvertFrom-Json
    $role.AssignableScopes = @($subScope)
    $rtmp = [System.IO.Path]::GetTempFileName()
    ($role | ConvertTo-Json -Depth 12) | Set-Content -Path $rtmp -Encoding utf8
    $exists = az role definition list --name $role.Name --query "[0].roleName" -o tsv
    if ($exists) { az role definition update --role-definition $rtmp | Out-Null }
    else { az role definition create --role-definition $rtmp | Out-Null }
    Remove-Item $rtmp -ErrorAction SilentlyContinue
}

# 6) Camada 2 - Deteccao em minutos (Action Group + Activity Log Alert).
# So aplica se -AlertEmail for informado. Dispara email quando um novo deployment de IA
# aparece no RG, antes do custo bater no Cost Management.
if ($AlertEmail) {
    Write-Host "==> Camada 2: deteccao (Action Group + Activity Log Alert)" -ForegroundColor Cyan
    $agName = 'ag-cost-guardrails'
    Invoke-Az @( 'monitor', 'action-group', 'create', '--name', $agName, '--resource-group', $ResourceGroup,
        '--short-name', 'CostGuard', '--action', 'email', 'devowner', $AlertEmail ) | Out-Null
    if (-not $DryRun) {
        $agId = az monitor action-group show --name $agName --resource-group $ResourceGroup --query id -o tsv
        Invoke-Az @( 'monitor', 'activity-log', 'alert', 'create', '--name', 'alert-ai-deployment-write',
            '--resource-group', $ResourceGroup, '--scope', $rgScope,
            '--condition', 'category=Administrative and
operationName=Microsoft.CognitiveServices/accounts/deployments/write',
            '--action-group', $agId,
            '--description', 'Camada 2: novo deployment de IA criado no RG sandbox.' ) | Out-Null
    }
    Write-Host " alerta ativo: email para $AlertEmail a cada novo deployment de IA." -ForegroundColor DarkGray
} else {
    Write-Host "==> Camada 2 IGNORADA. Passe -AlertEmail voce@empresa.com para criar o alerta de deteccao." -ForegroundColor Yellow
}

```

```
Write-Host ""
Write-Host "OK. Travas aplicadas em modo $Effect." -ForegroundColor Green
Write-Host "Passos manuais que faltam (ver guia, secao 9):" -ForegroundColor Green
Write-Host "  - Atribuir 'AI Sandbox Safe Dev' ao dev no RG e remover Owner/Contributor" -ForegroundColor Green
Write-Host "  - Zerar quota de PTU e de GPU vCPU na regioao $Location" -ForegroundColor Green
Write-Host "  - Fechar o bypass do Foundry: disableLocalAuth, publicNetworkAccess=Disabled, Private Endpoint" -ForegroundColor Green
Write-Host "  - Budget 50/75/90/100% + forecast -> Action Group -> Runbook" -ForegroundColor Green
```

scripts/verify-guardrails.ps1

```
#Requires -Version 7.0
<#
    verify-guardrails.ps1
    Mostra a compliance e PROVA que a trava de PTU bloqueia.
#>
[CmdletBinding()]
param(
    [Parameter(Mandatory)] [string] $SubscriptionId,
    [Parameter(Mandatory)] [string] $ResourceGroup,
    [string] $FoundryAccount
)
$ErrorActionPreference = 'Continue'
az account set --subscription $SubscriptionId | Out-Null
$rgScope = "/subscriptions/$SubscriptionId/resourceGroups/$ResourceGroup"

Write-Host "== Assignments no RG ==" -ForegroundColor Cyan
az policy assignment list --scope $rgScope --query "[].{name:name, policy:policyDefinitionId}" -o table

Write-Host "== Compliance (pode levar minutos para popular) ==" -ForegroundColor Cyan
az policy state summarize --resource-group $ResourceGroup -o json

Write-Host "== Camada 2: alertas de detecção no RG ==" -ForegroundColor Cyan
az monitor activity-log alert list --resource-group $ResourceGroup --query "[].{nome:name, habilitado:enabled}" -o table

if ($FoundryAccount) {
    Write-Host "== Teste vivo: criar um deployment PTU (esperado: BLOQUEADO) ==" -ForegroundColor Cyan
    az cognitiveservices account deployment create --name $FoundryAccount --resource-group $ResourceGroup --
    deployment-name test-ptu-should-fail --model-name gpt-4o --model-version "2024-08-06" --model-format OpenAI --
    sku-name ProvisionedManaged --sku-capacity 1 2>&1 | Out-Null
    if ($LASTEXITCODE -ne 0) { Write-Host "OK: bloqueado pela policy (esperado)." -ForegroundColor Green }
    else { Write-Host "ATENÇÃO: NÃO foi bloqueado. Confirme que -Effect Deny foi aplicado." -ForegroundColor Red }
}
}
```

scripts/teardown-guardrails.ps1

```
#Requires -Version 7.0
<#
    teardown-guardrails.ps1
    Remove tudo que o apply criou (rollback). Use o mesmo -Scenario do apply.
#>
[CmdletBinding()]
param(
    [Parameter(Mandatory)] [string] $SubscriptionId,
    [Parameter(Mandatory)] [string] $ResourceGroup,
    [ValidateSet('accountbased', 'hubbased')] [string] $Scenario = 'accountbased'
)
$ErrorActionPreference = 'Continue'
$subScope = "/subscriptions/$SubscriptionId"
$rgScope = "$subScope/resourceGroups/$ResourceGroup"
az account set --subscription $SubscriptionId | Out-Null

$assignments = @('allowed-types', 'allowed-locations', 'deny-localauth', 'restrict-network', 'require-
mi', 'approved-models', 'deny-sku-ptu')
if ($Scenario -eq 'hubbased') { $assignments += @('deny-gpu-compute', 'deny-serverless') }
foreach ($a in $assignments) {
    Write-Host "remove assignment: $a"
    az policy assignment delete --name $a --scope $rgScope 2>$null
}
}
```

```
$defs = @('deny-foundry-sku-ptu')
if ($Scenario -eq 'hubbased') { $defs += @('deny-ml-gpu-compute','deny-ml-serverless-mktplace') }
foreach ($d in $defs) {
    Write-Host "remove definition: $d"
    az policy definition delete --name $d --subscription $SubscriptionId 2>$null
}

Write-Host "remove custom role 'AI Sandbox Safe Dev' (so funciona se nao houver atribuicoes ativas)"
az role definition delete --name 'AI Sandbox Safe Dev' 2>$null
Write-Host "Rollback concluido."
```

11. As policies e o custom role

policies/definitions/deny-deployment-sku.json (a trava central)

Esta e a unica peca sem equivalente built-in. E uma allow-list de SKUs de consumo: tudo que nao for Standard/GlobalStandard/DataZoneStandard (ou seja, PTU/ProvisionedManaged) e negado.

```
{
  "properties": {
    "displayName": "Deny Foundry deployment com SKU provisionado (mata PTU)",
    "policyType": "Custom",
    "mode": "All",
    "description": "Nega criacao de deployments cujo SKU nao esteja na lista permitida. Bloqueia ProvisionedManaged / PTU, que cobra capacidade reservada por hora independente de token.",
    "metadata": { "category": "Cost Governance", "version": "1.0.0" },
    "parameters": {
      "allowedSkus": {
        "type": "Array",
        "metadata": { "displayName": "SKUs permitidos (apenas consumo por token)" },
        "defaultValue": [ "Standard", "GlobalStandard", "DataZoneStandard" ]
      },
      "effect": {
        "type": "String",
        "allowedValues": [ "Deny", "Audit", "Disabled" ],
        "defaultValue": "Deny"
      }
    },
    "policyRule": {
      "if": {
        "allOf": [
          { "field": "type", "equals": "Microsoft.CognitiveServices/accounts/deployments" },
          { "not": { "field": "Microsoft.CognitiveServices/accounts/deployments/sku.name", "in": "[parameters('allowedSkus')]" } }
        ]
      },
      "then": { "effect": "[parameters('effect')]" }
    }
  }
}
```

rbac/role-ai-sandbox-safe-dev.json

O AssignableScopes vem como placeholder; o apply injeta a sua subscription em tempo de execucao. Pense nesta role como a persona do dev: tudo que ele PODE esta em Actions, tudo que ele NAO pode esta em NotActions.

PODE (Actions)	NAO PODE (NotActions)
Ler o RG e gerenciar deployments ARM	Escrever ou apagar RBAC e elevar acesso (Authorization/*)
Ler a conta do Foundry e seus modelos	Mexer em Quota, Capacity, Commerce e Billing
Criar, ler e apagar deployments de modelo (Standard)	Alterar ou apagar a conta do Foundry (accounts write/delete)
Ler telemetria (App Insights / Log Analytics)	Pegar a chave: listKeys / regenerateKey (forca uso via gateway)

Atencao ao limite do modelo: NotActions NAO e um deny. Ele so subtrai o que ESTA role concede. Se o dev tambem for Owner ou Contributor (direto ou herdado de cima), os NotActions nao valem. Por isso o Passo 6 insiste em deixar SO esta role no RG e checar heranca.

```
{
  "Name": "AI Sandbox Safe Dev",
  "IsCustom": true,
  "Description": "Dev de sandbox de IA. Cria e gerencia deployments Standard no Foundry pre-provisionado e le observabilidade. NAO mexe em RBAC, quota, capacity, billing, nem altera a conta do Foundry. Sem listKeys: forca uso via gateway.",
  "Actions": [
    "Microsoft.Resources/subscriptions/resourceGroups/read",
    "Microsoft.Resources/deployments/*",
    "Microsoft.CognitiveServices/accounts/read",
    "Microsoft.CognitiveServices/accounts/models/read",
    "Microsoft.CognitiveServices/accounts/deployments/read",
    "Microsoft.CognitiveServices/accounts/deployments/write",
    "Microsoft.CognitiveServices/accounts/deployments/delete",
    "Microsoft.Insights/components/read",
    "Microsoft.Insights/components/query/read",
    "Microsoft.OperationalInsights/workspaces/read"
  ],
  "NotActions": [
    "Microsoft.Authorization/*/write",
    "Microsoft.Authorization/*/delete",
    "Microsoft.Authorization/elevateAccess/action",
    "Microsoft.Quota/*",
    "Microsoft.Capacity/*",
    "Microsoft.Commerce/*",
    "Microsoft.Billing/*",
    "Microsoft.CognitiveServices/accounts/write",
    "Microsoft.CognitiveServices/accounts/delete",
    "Microsoft.CognitiveServices/accounts/listKeys/action",
    "Microsoft.CognitiveServices/accounts/regenerateKey/action"
  ],
  "AssignableScopes": [ "/subscriptions/<SUA_SUBSCRIPTION_ID>" ]
}
```


policies/params/allowed-resource-types.accountbased.json

A trava de maior impacto. So estes tipos podem ser criados no RG. Ajuste conforme o que o dev legitimamente precisa (inclua Microsoft.Network se for usar Private Endpoint).

```
{
  "listOfResourceTypesAllowed": {
    "value": [
      "microsoft.cognitiveservices/accounts",
      "microsoft.cognitiveservices/accounts/deployments",
      "microsoft.storage/storageaccounts",
      "microsoft.insights/components",
      "microsoft.insights/actiongroups",
      "microsoft.operationalinsights/workspaces",
      "microsoft.network/privateendpoints",
      "microsoft.network/privatednszones"
    ]
  }
}
```

12. Apendice: policias built-in usadas

IDs de built-in do Azure são globais (iguais em qualquer tenant), por isso já vem no script. So a trava de SKU/PTU e custom.

Controle	Built-in DefinitionId	Effects
Allowed resource types	a08ec900-254a-4555-9bf5-e42af04b5c5c	Deny
Allowed locations	e56962a6-4747-49cd-b67b-bf8b01975c4c	Deny
Key access disabled (disableLocalAuth)	71ef260a-8f18-47b7-abcb-62d0673d94dc	Audit/Deny/Disabled
Restrict network access	037eea7a-bd0a-46c5-9a66-03aea78705d3	Audit/Deny/Disabled
Approved models (opcional)	aafe3651-cb78-4f68-9f81-e7e41509110f	Audit/Deny/Disabled
Managed identity	fe3fd216-4f83-4fc1-8984-2bbec80a3418	Audit/Deny/Disabled

Apendice B: custom de hub-based (deny GPU compute)

So necessaria se o Foundry for hub-based (Microsoft.MachineLearningServices/workspaces).

```
{
  "properties": {
    "displayName": "Deny Azure ML compute com VM GPU (hub-based)",
    "mode": "All",
    "parameters": {
      "deniedVmPatterns": { "type": "Array",
        "defaultValue": [ "Standard_NC*", "Standard_ND*", "Standard_NV*", "Standard_NG*" ] },
      "effect": { "type": "String", "allowedValues": ["Deny","Audit","Disabled"], "defaultValue": "Deny" }
    },
    "policyRule": {
      "if": { "allOf": [
        { "field": "type", "equals": "Microsoft.MachineLearningServices/workspaces/computes" },
        { "anyOf": [
          { "count": { "value": "[parameters('deniedVmPatterns')]", "name": "p",
            "where": { "field": "Microsoft.MachineLearningServices/workspaces/computes/Am1Compute.vmSize",
              "like": "[current('p')]" } }, "greater": 0 },
          { "count": { "value": "[parameters('deniedVmPatterns')]", "name": "p2",
            "where": { "field": "Microsoft.MachineLearningServices/workspaces/computes/ComputeInstance.vmSize", "like": "[current('p2')]" } }, "greater": 0 }
        ] }
      ] },
      "then": { "effect": "[parameters('effect')]" }
    }
  }
}
```

Aviso

Este material é um padrão de referência. Teste sempre em ambiente não produtivo e aplique em modo Audit antes de Deny. O Azure não oferece parada automática de gasto por valor em um Resource Group; o teto em dólar é obtido pela combinação das quatro camadas (prevenção na origem + quota de token no gateway + budget reativo), não por um único controle. Valide os aliases de policy no seu ambiente (az policy alias list) antes de promover para Deny.